LEBANESE INTERNATIONAL UNIVERSITY
Department of Computer Science and Information Technology

CSCI426 · Advanced Web Programming

Project Phase 1

HealthTrack

A Fitness and Wellness Tracking Web Application



Author	Ali Mostafa
Student ID	12431380
Instructor	Dr. Iman Darwish
Theme	6, HealthTrack (Fitness & Wellness)
Live demo	https://healthtrack-1.netlify.app
GitHub	https://github.com/alimoustapha/healthtrack
Submitted	November 2026

Contents

1. Abstract

2. Introduction and Problem

3. System Design

4. Technologies Used

5. User Interface

6. Key Code Snippets

7. Responsive Design

8. Version Control and Deployment

9. Conclusion

1. Abstract

For this project I built HealthTrack, a React website where you can write down what you eat and how much you move during the day. Meals are logged in calories, and exercise and other activities in minutes. The site adds them up so you can see today's totals at a glance.

I also added six workout plans. You can search them or filter them by type, and each one has its own page (a dynamic route) with the list of exercises. One button adds the workout to your log for today.

The site has seven pages, a light and a dark theme, and forms that check the input before saving it. It works on a laptop and on a phone. Everything is saved in the browser with localStorage, so there is no backend and no sign-up. The code is on GitHub and the site is live on Netlify.

2. Introduction and Problem

Many students want to eat better or train more, but they lose track. They forget what they ate at lunch, how long they were at the gym, or which exercises to do. A lot of fitness apps also ask you to create an account or pay before you can use the main features.

So I wanted something simple and free: open the site, add a meal or a workout in a few seconds, and see the total for the day straight away. My goals were to:

- make adding a meal, a workout or an activity quick,
- show a daily summary with calories eaten, active minutes and the number of logs,
- give beginners some ready workout plans with sets and reps,
- keep the design clean, work well on phones, and include a dark mode.

3. System Design

3.1 Component Architecture

The app starts in index.js, which renders App. I kept the theme state in App because both the Navbar and the whole page need it. App also holds the Footer and the Router, and the Router decides which page to show for the current URL. Two small components, StatCard and WorkoutCard, are reused on more than one page. The workout plans come from a JSON file, and the user's logs come from localStorage.

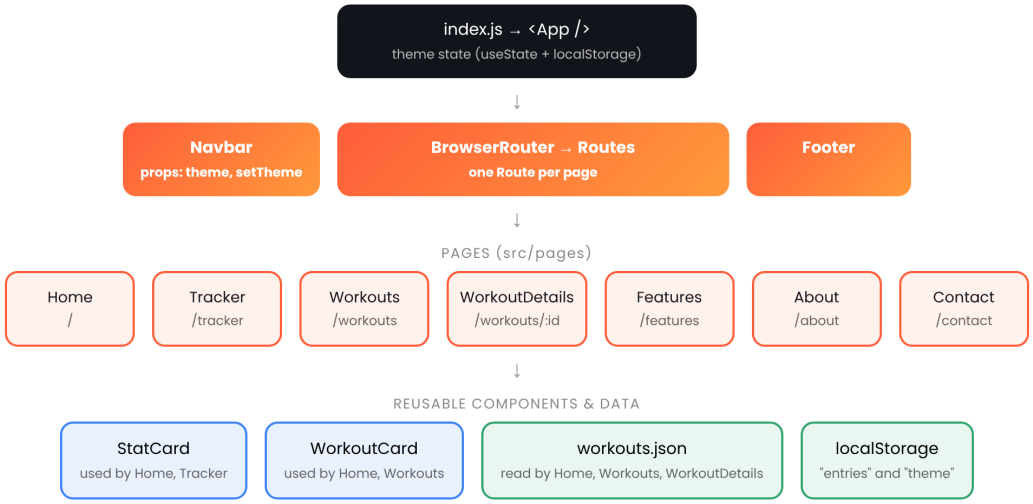


Figure 1: Component tree of HealthTrack

3.2 Pages and Routes

Route	Page	Purpose
/	Home	Welcome section, today's summary, three popular workouts and a button to start logging
/tracker	Tracker	Form to add a log, the list of logs with filters and delete, and today's totals
/workouts	Workouts	All workout plans, with a search box and category buttons
/workouts/:id	WorkoutDetails	The dynamic page: shows the workout whose id is in the URL
/features	Features	Six cards explaining what the app does
/about	About	Why I made it, who it is for, and how it works in 3 steps
/contact	Contact	Contact details and a validated message form
*	NotFound	404 page for any wrong link

3.3 Data Model

Each thing the user logs is saved as an object like this, in a list stored in localStorage under the key `entries`:

Field	Type	Example
id	number	1762889000000 (from Date.now(), unique)
type	text	"Meal", "Exercise" or "Activity"
name	text	"Chicken salad"
value	number	450 (kcal for meals, minutes for the others)
date	text	"2026-11-11"

I used Date.now() for the id because it gives a different number every time, which is enough for a list that only lives in one browser. The workouts are in src/data/workouts.json. Each one has an id, name, category, level, minutes, calories, a short description and its exercises (name, sets and reps).

3.4 What Happens When You Add a Log

When the form is submitted, the app checks the inputs first. If something is missing or wrong it shows an error and stops. Otherwise it adds the new log to the state, React redraws the list, and a useEffect saves the updated list to localStorage. Home reads the same list, so its summary updates too.

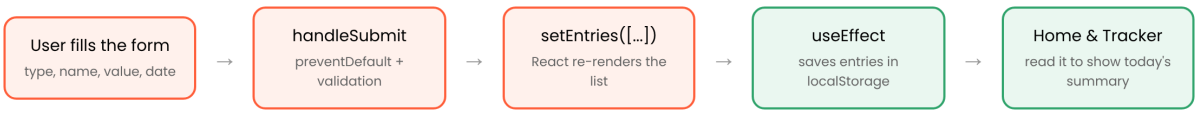


Figure 2: Adding a log, step by step

4. Technologies Used

Technology	Used for
ReactJS 18 (Create React App)	The whole interface, split into pages and small components
React Router DOM 6	Moving between pages without reloading, and reading the id in /workouts/:id with useParams
useState and useEffect	useState for the form inputs, filters, theme and phone menu; useEffect to save to localStorage
localStorage	Keeping the logs and the chosen theme after a refresh
JSON	The six workout plans (src/data/workouts.json)
CSS	Plain CSS: variables for the two themes, Flexbox, Grid, hover effects and media queries for phones
Poppins (Google Fonts)	The font used across the site
Git and GitHub	Saving my progress in commits and sharing the code
Netlify	Hosting the live site
VS Code	Writing the code

5. User Interface

The colors are a dark navy (#12141b) and an orange gradient (#ff5e3a to #ff9a3c). Cards have rounded corners and a light shadow, and they lift a little when you hover over them. The navbar stays at the top while you scroll, and the page you are on is highlighted in orange.

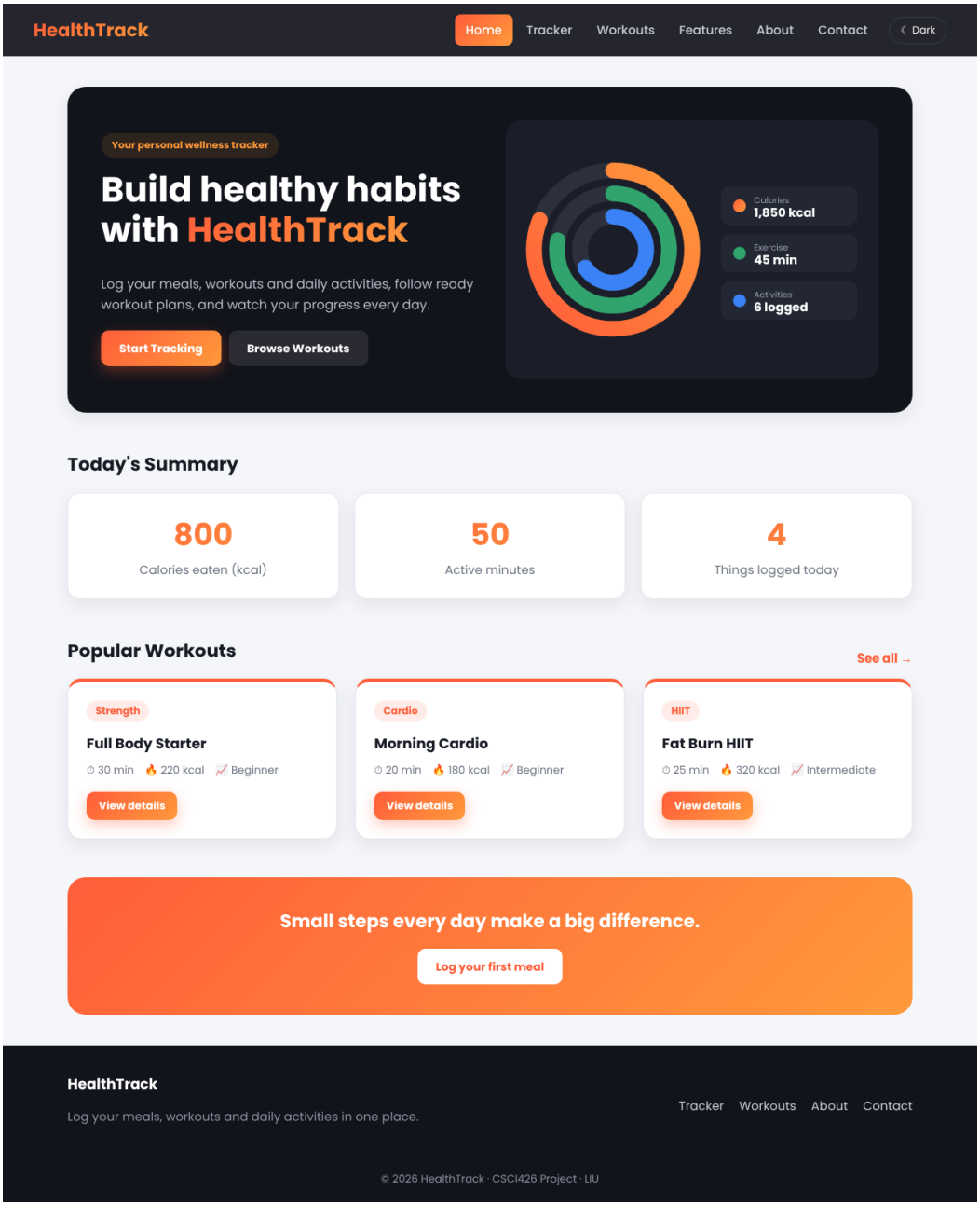


Figure 3: Home page

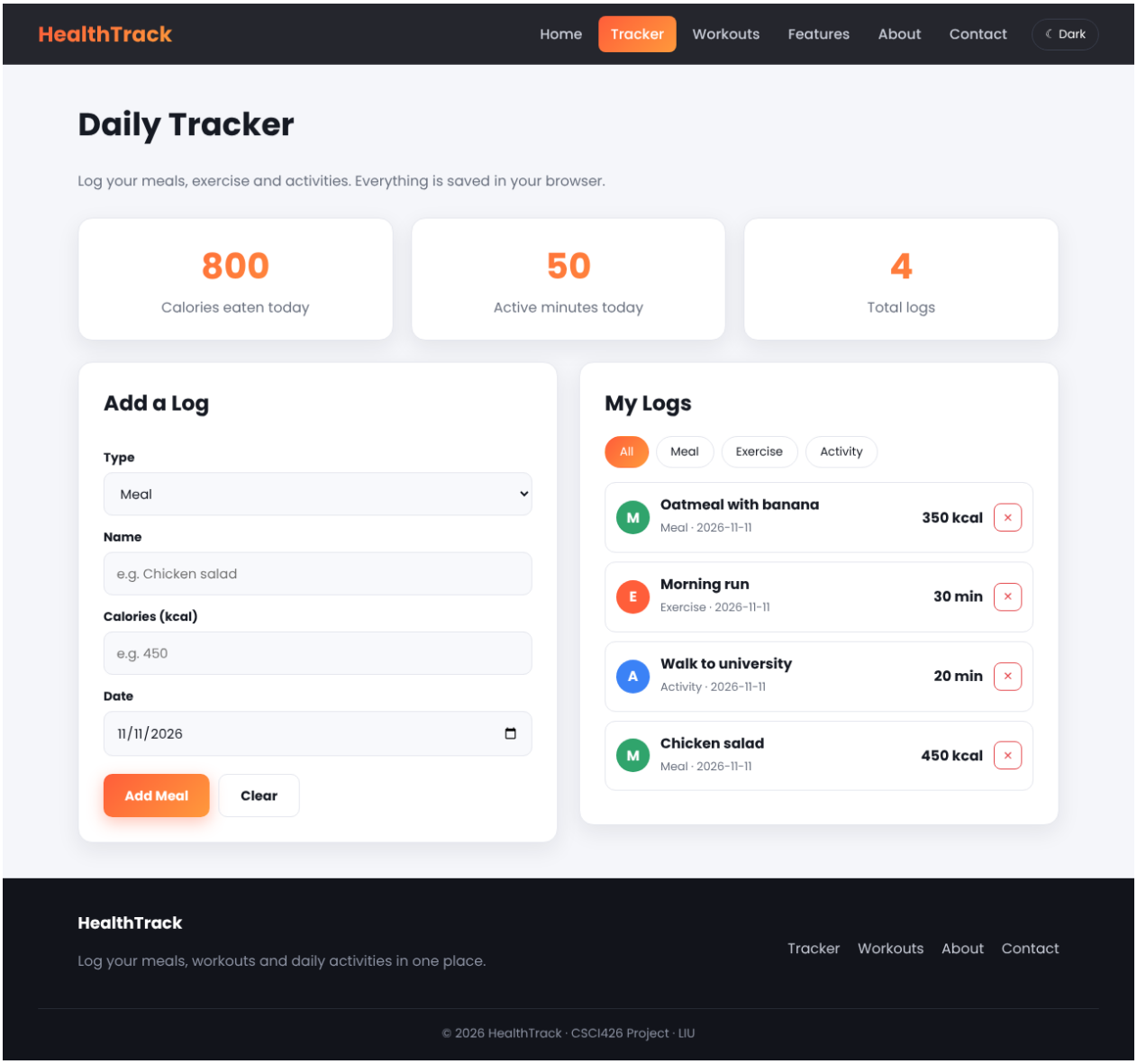


Figure 4: Daily Tracker

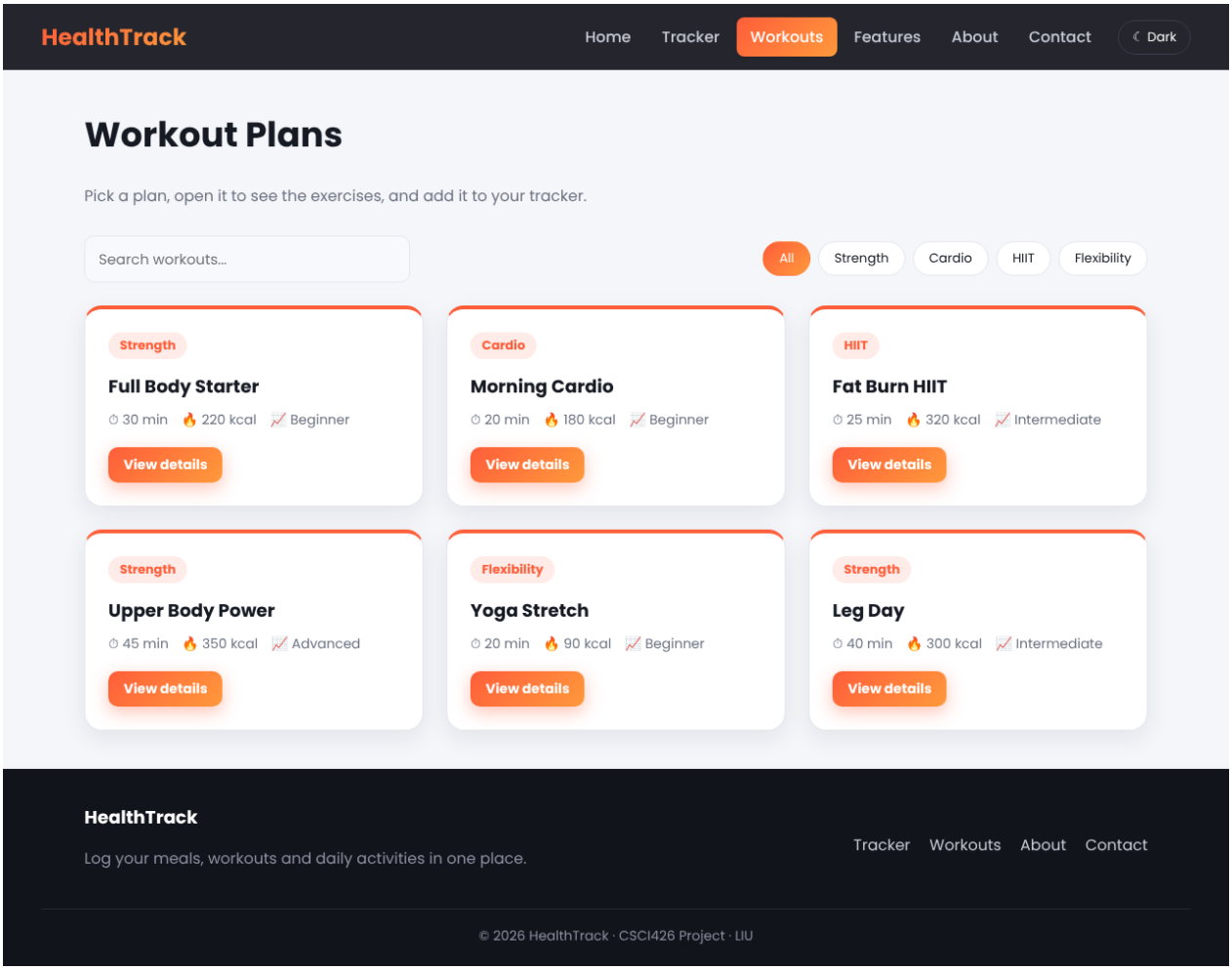


Figure 5: Workout plans with search and filters

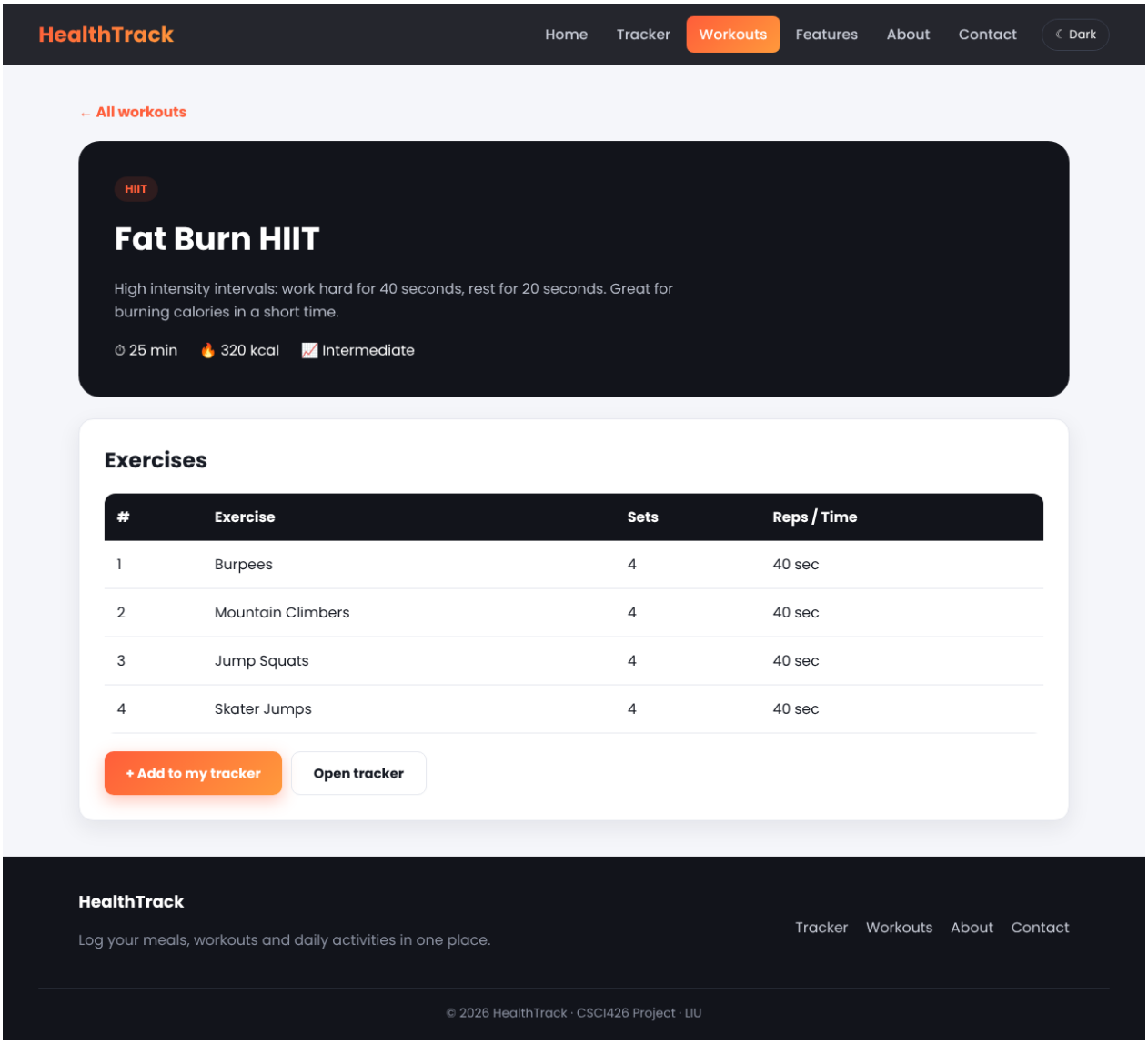


Figure 6: Workout details, the dynamic page (/workouts/3)

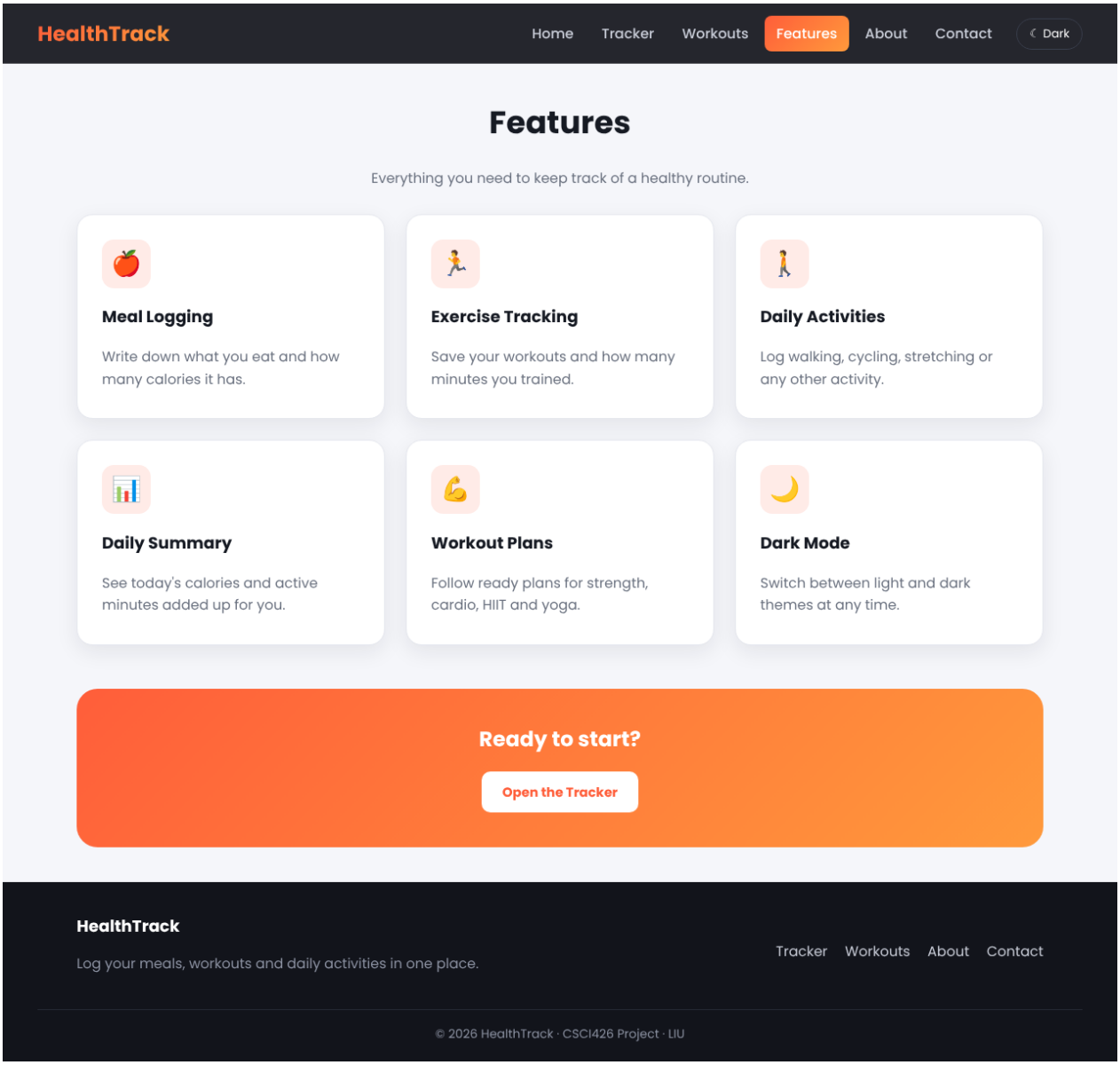


Figure 7: Features

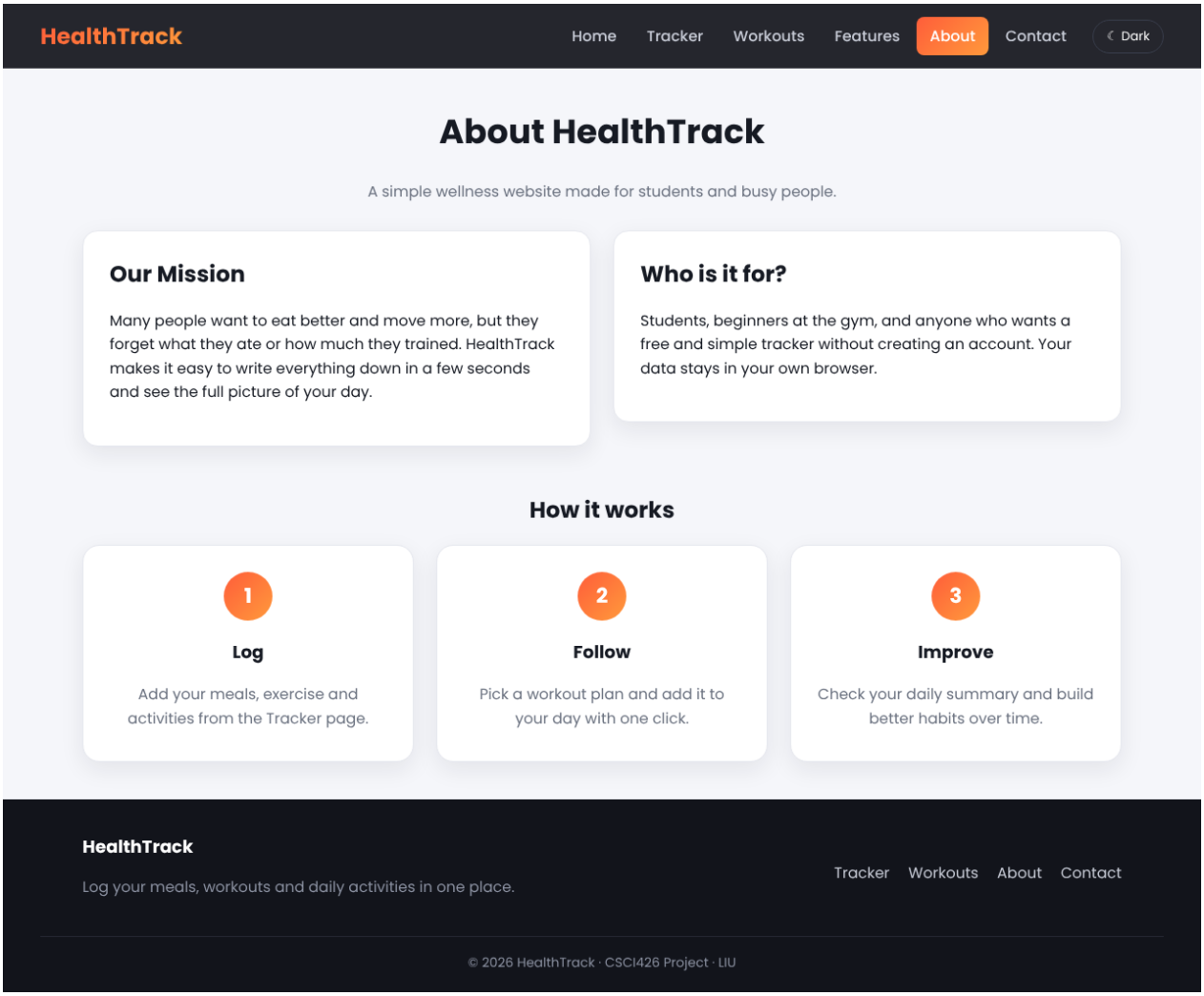


Figure 8: About

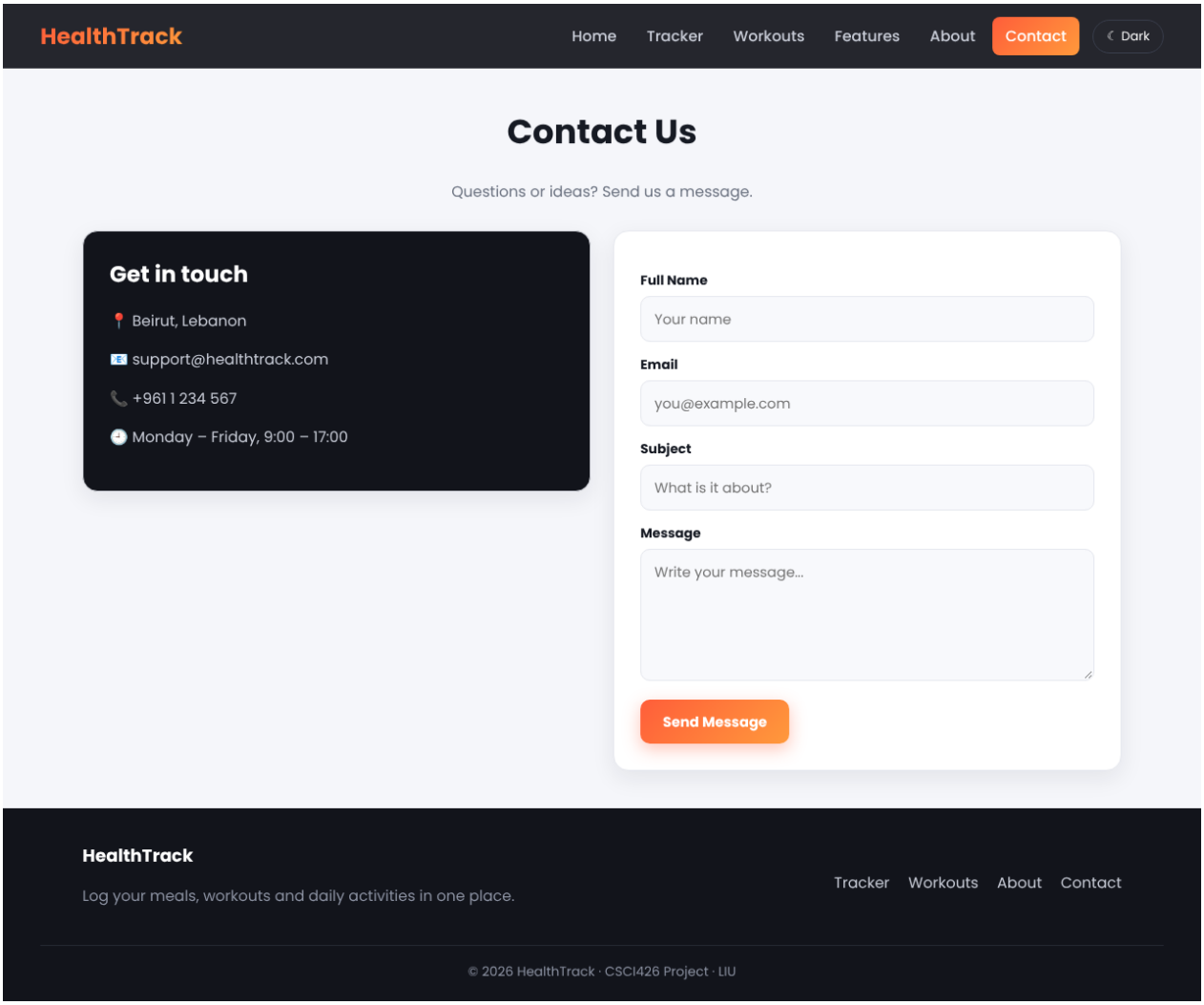


Figure 9: Contact page

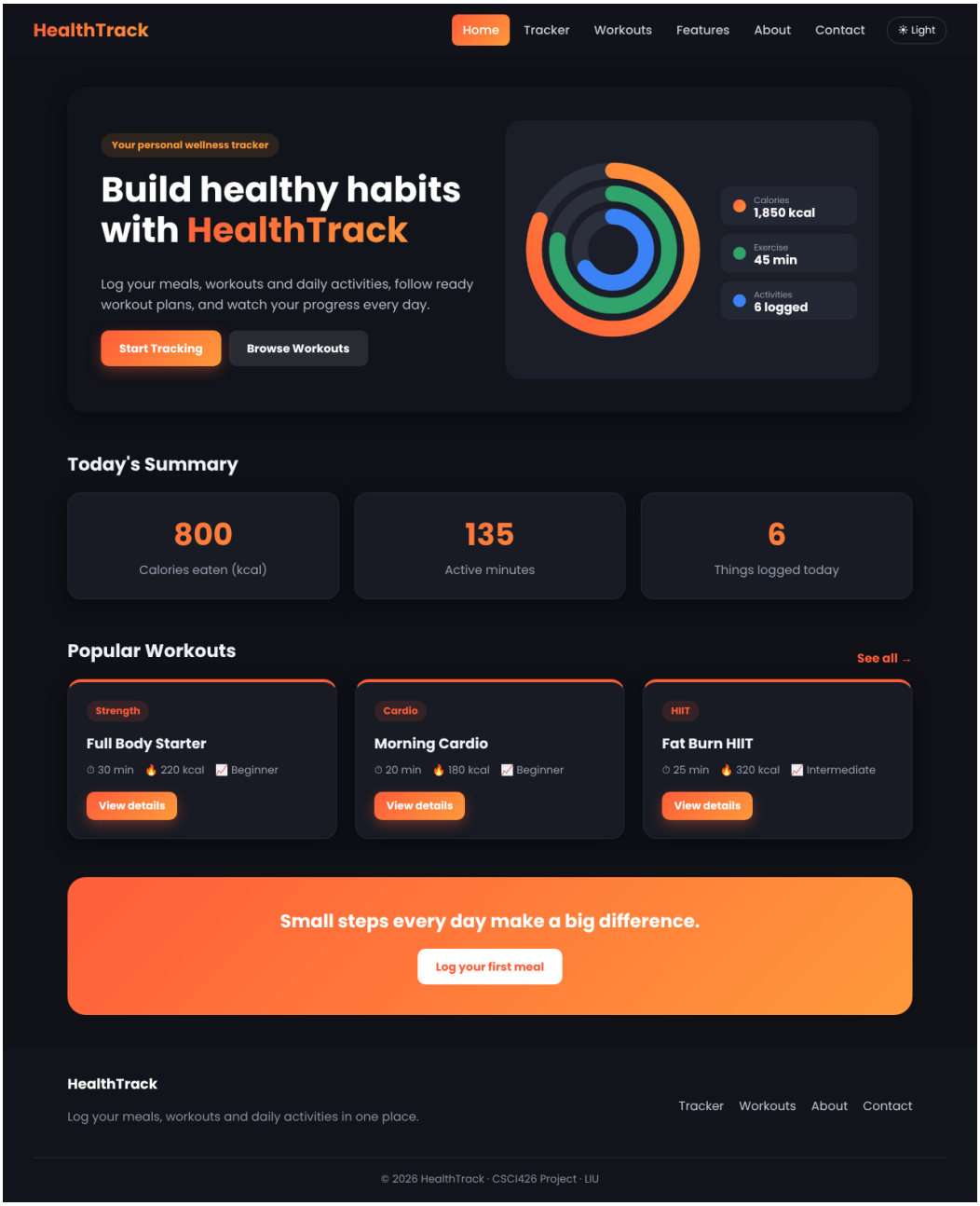


Figure 10: Home page in dark mode

6. Key Code Snippets

6.1 Routing (App.js)

Every page gets its own Route. The `:id` in the workouts route is what makes that page dynamic, and the * route at the end catches any link that doesn't exist. I also save the theme with useEffect so dark mode is still on after a refresh.

src/App.js

```
const [theme, setTheme] = useState(localStorage.getItem("theme") || "light");

useEffect(() => {
  localStorage.setItem("theme", theme);
}, [theme]);

<div className="App" data-theme={theme}>
  <Router>
    <Navbar theme={theme} setTheme={setTheme} />
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/tracker" element={<Tracker />} />
      <Route path="/workouts" element={<Workouts />} />
      <Route path="/workouts/:id" element={<WorkoutDetails />} />
      <Route path="/features" element={<Features />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  </Router>
</div>
```

6.2 Dynamic Page (WorkoutDetails.js)

useParams gives me the id from the URL as text, so I convert it with Number() before searching the JSON list with find. If no workout matches, the page shows a "not found" message instead of crashing.

src/pages/WorkoutDetails.js

```
const { id } = useParams();
const workout = workouts.find((w) => w.id === Number(id));

if (!workout) {
  return <h1>Workout not found</h1>;
}

{workout.exercises.map((ex, i) => (
  <tr key={i}>
    <td>{i + 1}</td>
    <td>{ex.name}</td>
    <td>{ex.sets}</td>
    <td>{ex.reps}</td>
  </tr>
))}
```

6.3 Saving Logs and Validating the Form (Tracker.js)

The list starts from whatever is in localStorage, or from a few demo logs the first time. Each check in handleSubmit stops the function early with an error, so a log is only saved when everything is valid.

src/pages/Tracker.js

```
const [entries, setEntries] = useState(
  JSON.parse(localStorage.getItem("entries")) || demoEntries
);

useEffect(() => {
  localStorage.setItem("entries", JSON.stringify(entries));
}, [entries]);

const handleSubmit = (e) => {
  e.preventDefault();
  setSuccess("");

  if (!name || !value || !date) {
    setError("Please fill in all the fields.");
    return;
  }
  if (value <= 0) {
    setError("The number must be bigger than 0.");
    return;
  }
  const newEntry = {
    id: Date.now(),
    type: type,
    name: name,
    value: Number(value),
    date: date,
  };
  setEntries([newEntry, ...entries]);
  setSuccess(`${type} "${name}" was added.`);
};

const handleDelete = (id) => {
  setEntries(entries.filter((e) => e.id !== id));
};
```

6.4 Search and Filter (Workouts.js)

A workout is shown only if its name contains the search text and its category matches the selected button.

src/pages/Workouts.js

```
const shown = workouts.filter((w) => {
  const matchName = w.name.toLowerCase().includes(search.toLowerCase());
  const matchCategory = category === "All" || w.category === category;
  return matchName && matchCategory;
});
```

6.5 Reusable Component with Props (WorkoutCard.js)

I wrote the card once and use it on both Home and Workouts. The page passes in the data as props.

src/components/WorkoutCard.js

```
const WorkoutCard = ({ id, name, category, level, minutes, calories }) => {
  return (
    <div className="card workout-card">
      <span className="tag">{category}</span>
      <h3>{name}</h3>
      <div className="workout-info">
        <span>{minutes} min</span>
        <span>{calories} kcal</span>
        <span>{level}</span>
      </div>
      <Link to={`/workouts/${id}`} className="btn btn-small">View details</Link>
    </div>
  );
};
```

6.6 Light and Dark Themes (App.css)

All the colors come from CSS variables, so I only had to define them twice. When the button changes `data-theme` on the main div, the whole site switches colors at once.

src/App.css

```
.App[data-theme="light"] { --bg: #f5f6fa; --box: #ffffff; --text: #161a23; }
.App[data-theme="dark"]  { --bg: #0f1117; --box: #181b24; --text: #f2f3f7; }

.App {
  background: var(--bg);
  color: var(--text);
  transition: background 0.3s, color 0.3s;
}
```

7. Responsive Design

The cards use CSS Grid with `repeat(auto-fill, minmax(260px, 1fr))`, so they drop to fewer columns by themselves when the screen gets narrower. Other rows use Flexbox with `flex-wrap`.

Six links don't fit in a phone's navbar, so under 900px a media query hides them behind a menu button, and clicking it flips a `menuOpen` state in the Navbar.

src/App.css (media query)

```
@media (max-width: 900px) {
  .menu-btn { display: block; }
  .nav-links { display: none; }
  .nav-links.open { display: flex; }
}
```

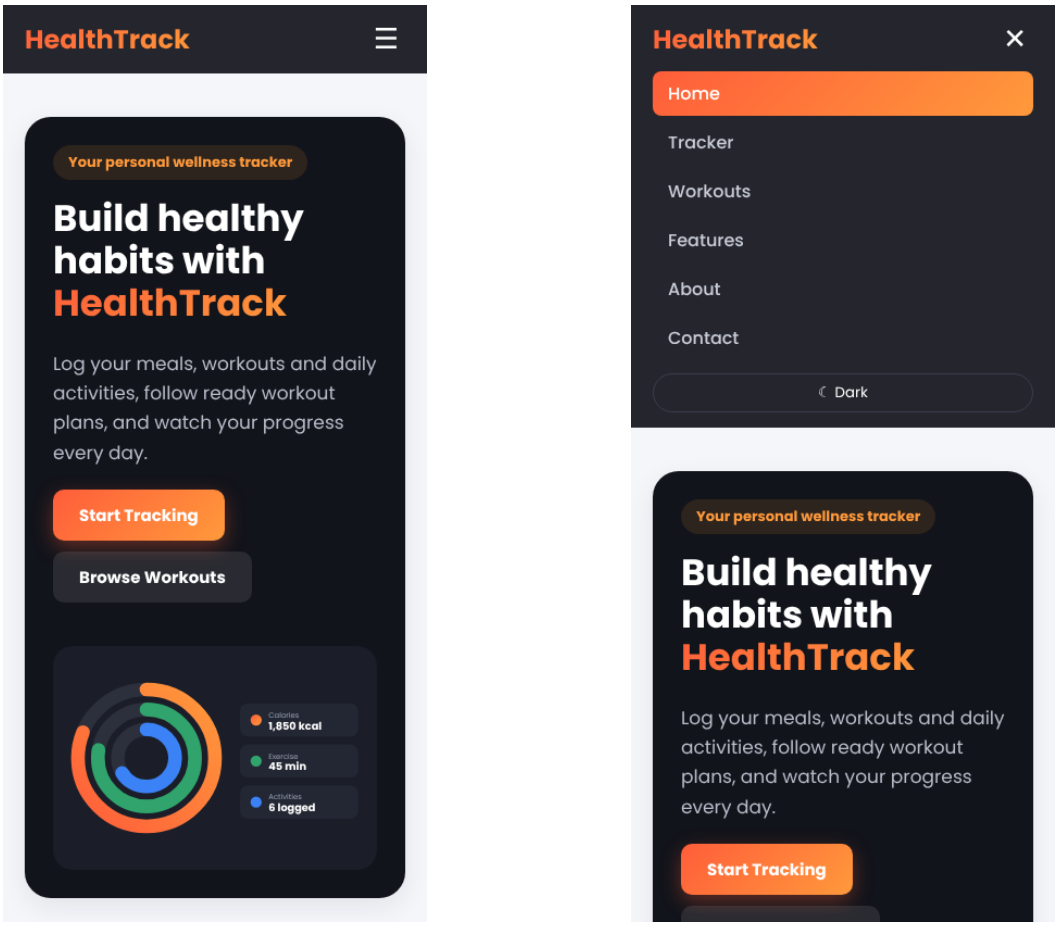


Figure 11: The site on a phone (390px wide), with the menu closed and open

8. Version Control and Deployment

The project is in Git, committed in small steps (setup, components, pages, styling, then the README), so the history shows how the project grew. The code is on GitHub at `github.com/alimoustapha/healthtrack`.

The live site is on Netlify at `healthtrack-1.netlify.app`. Netlify builds it again on its own every time I push to GitHub. Without extra setup, refreshing a page like /workouts/3 would give a 404, because Netlify would look for a real file with that name. A `_redirects` file that sends every URL to index.html prevents that. I also added a `vercel.json` that does the same on Vercel.

The README explains what the project is, how to run it locally, and has screenshots of every page.

9. Conclusion

HealthTrack covers what the project asked for: it is built in React, has more than five pages including a dynamic one, works on phones and laptops, and is on GitHub and live online. It also made me use most of what we covered in class: components and props, useState and useEffect, React Router and useParams, form validation, map and filter on lists, localStorage, and CSS variables for theming.

If I keep working on it, I would add a backend with user accounts so the data isn't stuck in one browser, weekly charts to see progress, and tracking for water and sleep.